

Reinforcement Learning-Based Dynamic Pricing for Ridesharing Revenue Optimisation

Project Report

Ayodeji Ali

25168096

Background

Dynamic pricing is a pricing strategy where the price changes depending on the market conditions, for example how many people are using one service, how many spaces are available in this service and how many workers available to produce this service, etc. Depending on the markets conditions it allows the service price to increase or decrease.

Businesses will use dynamic pricing increase their revenue when the demand is high and to attract customers when the demand is low. It's mostly used in ridesharing apps, hotels, airlines, retail and commerce. In ridesharing companies like Uber are known to increase the prices when the demand is high or when there aren't many drivers available. So, the aim is to balance the customers demand, the available supply and the business's profit. This is why reinforcement learning is a perfect use for this dilemma as pricing is a repeated decision-making problem. The agent learns from rewards and uses that to improve its pricing overtime.

Dataset:

I dataset I am using is a dynamic pricing dataset that I had downloaded from Kaggle. The dataset represent data from a ridesharing app / transportation business. It contains information about ride demand, driver supply, customer behaviour and ride cost. The Important features are number of riders, number of drivers, location category, customer loyalty status, number of past rides, average ratings, time of booking, vehicle type, expected ride duration and historical cost of ride.

The main pricing related target is the historical cost of ride. The supply and demand can be represented by using the number of riders and number of drivers. This is why the dataset is suitable because it already contains all the type of information that is needed to create dynamic pricing. It allows the model to project how the different pricing decisions can affect a company's revenue stream.

RL Formulation

State

The state represents the market situation at a given time. In the project the state includes the features that were previously mentioned within the Dataset section of the report. The features indicate to the model whether the market is busy, normal or quiet. Like how high riders and low drivers might suggest a higher demand pressure.

Action

Action is the pricing decision that's chosen by the reinforcement learning agent. The agent chooses a price multiplier. If the multiplier is below 1.0 then it represents the market value as 1.0 represents the base or the normal price. A multiplier above 1.0 represents an increase in the price, this would be considered a surge in the instance of dynamic pricing.

Reward

The reward is mainly based on revenue. A simple reward function works as so:

$$Reward = AdjustedPrice \times PredictedDemand$$

The adjusted price is calculated as follows:

$$AdjustedPrice = HistoricalRideCost \times PriceMultiplier$$

A penalty can be added if the penalty is too high, if the price is too high it can scare away customers. The penalty will prevent the agents from always choosing the highest price. For example:

$$Reward = Revenue - Penalty$$

The penalty represents possible customer dissatisfaction or demand loss from excessive pricing.

Environment

The environment is built using the historical dataset. The data is used to simulate a ridesharing applications pricing outcomes as the agent can't interact with a real ridesharing platform. Machine learning models can be trained to predict demand and expected ride cost from the available features. The Reinforcement Learning agent then tests the different pricing actions inside the simulated environment created and the environment returns a reward each time.

Algorithms

I used Q-Learning as the main Reinforcement Learning approach for this project. It's suitable because the pricing action are discrete and simple. The agent chooses from fixed prices multipliers like 0.8, 0.9, 1.0, 1.1, 1.2 and 1.3. The agent learns which multiplier gives the highest reward in different market states.

A Random Forest Regression model is used to support the simulated environment. The model predicts the expected base ride cost from the cleaned ride and market features. This predicted cost is then used in the reward calculation when the RL agent applies different pricing multipliers.

Random Forest was selected because it can handle non-linear relationships between features such as rider demand, driver supply, ride duration, customer loyalty, booking time and vehicle type. It also works well with mixed numerical and categorical data after preprocessing.

Two baseline methods are used for comparison. The first is a fixed-price baseline, where the multiplier is always 1.0. The second is a rule-based pricing baseline, where the price increases when demand is high compared with supply and decreases when demand is low. The RL pricing strategy is then compared against these simpler approaches using total revenue, average reward, retained demand and average selected multiplier.

Code Implementation

I used python using jupyter notebook for the code implementation. It's divided into 4 main notebooks so that each stage of the project can be displayed clearly and is easily legible: data preparation, Random Forest training, Q-learning training, and final policy evaluation.

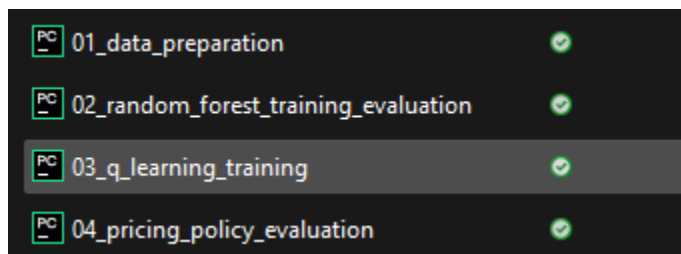


Figure 1: Notebook workflow

The first notebook deals with that cleaning and preparation of the data. It loads the `dynamic_pricing.csv` dataset, checks the dataset shape, data types, missing values, duplicate rows and summary statistics. The column names are cleaned into a consistent format, duplicate records are removed, and invalid numeric values are handled. New features are also created, including `demand_supply_ratio`, `riders_per_minute` and `experienced_customer`. The now cleaned data is split into training and testing, and the sets are saved for the next notebooks.

The second notebook trains the Random Forest Regression model. A preprocessing pipeline handles both numerical and categorical variables. The numerical features and input a scaled and categorical features like location and loyalty status, booking time and vehicle type are encoded. The Random Forest model is trained to predict the expected base ride cost, using `historical_cost_of_ride` as the target variable. The model is evaluated using MAE, RMSE and R^2 .

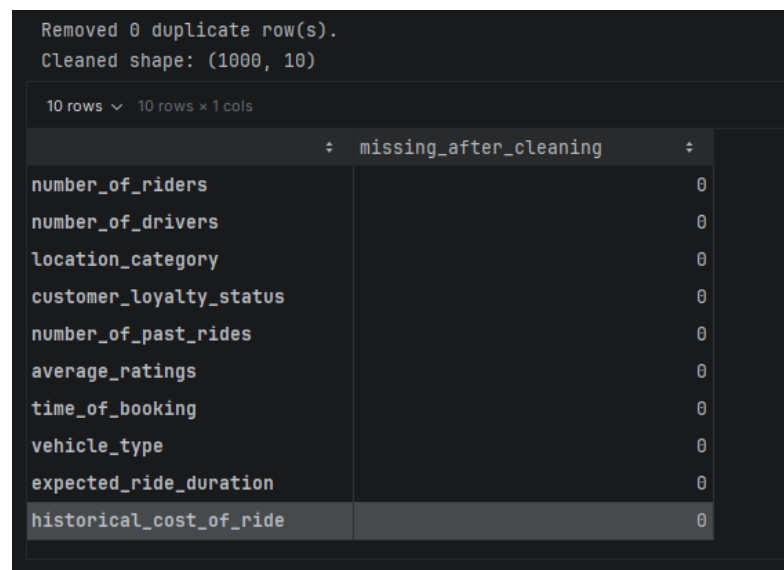
The third notebook implements the Q-learning dynamic pricing agent. The state is created from market, and ride features such as number of riders, number of drivers, demand-supply ratio, ride duration, customer history, rating, loyalty, location, booking time and vehicle type. The action space is defined as price multipliers: 0.8, 0.9, 1.0, 1.1, 1.2 and 1.3. A reward function is used to calculate adjusted revenue while applying a penalty for excessive pricing. Continuous state values are placed into bins so that they can be used in a Q-table.

The fourth notebook evaluates the pricing strategies on the test set. Three methods are compared: fixed pricing, rule-based pricing and RL pricing. Fixed pricing always uses a multiplier of 1.0. Rule-based pricing changes the multiplier depending on the demand-supply ratio. RL pricing selects the multiplier with the highest learned Q-value for each market state. The final results compare total expected revenue, average reward, average multiplier and retained demand.

The implementation follows the full dynamic pricing workflow: data cleaning, feature engineering, environment modelling, Q-learning training and policy evaluation. This allows the RL pricing strategy to be compared against simpler pricing methods in a clear and reproducible way.

Results and Evaluation

I compared three pricing strategies fixed pricing, rule-based pricing and Reinforcement Learning pricing. Fixed stays using a constant multiplier of 1.0 throughout every ride, Rule based uses adjustments in the multiplier based a supply-demand ratio, RL model selects the multiplier with the highest Q value for each market.



Removed 0 duplicate row(s).
Cleaned shape: (1000, 10)

10 rows ▾ 10 rows × 1 cols

	missing_after_cleaning
number_of_riders	0
number_of_drivers	0
location_category	0
customer_loyalty_status	0
number_of_past_rides	0
average_ratings	0
time_of_booking	0
vehicle_type	0
expected Ride duration	0
historical_cost_of_ride	0

Figure 2: Cleaned dataset and missing value check

5 rows × 13 cols										Static Output		
	numb...	number_of...	location_category	custome...	number_o...	average_ratings	time_of_booking	vehicle_type	expected...	historical_cost...	demand_su...	
0	90	45	Urban	Silver	13	4.47	Night	Premium	90	284.257273	2.000000	
1	58	39	Suburban	Silver	72	4.06	Evening	Economy	43	173.874753	1.487179	
2	42	31	Rural	Silver	0	3.99	Afternoon	Premium	76	329.795469	1.354835	
3	89	28	Rural	Regular	67	4.31	Afternoon	Premium	134	470.201232	3.178571	
4	78	22	Rural	Regular	74	3.77	Afternoon	Economy	149	579.681422	3.545455	

2 rows × 8 cols										Static Output		
	count	mean	std	min	25%	50%	75%	max				
demand_supply_ratio	1000.0	3.235461	2.533519	1.112360	1.658793	2.357143	3.800000	17.6				
riders_per_minute	1000.0	0.994620	1.169348	0.116667	0.375414	0.598165	1.052529	9.1				

Figure 3: Feature engineering output and demand-supply statistics

Random forest model was first evaluated as it provides the expected base ride-cost estimate used in the simulated pricing environment. The model was evaluated using MAE, RMSE and R^2 . It achieved an MAE of 55.60, an RMSE of 74.44 and an R^2 score of 0.848 on the test set. The R^2 score shows that the model explained a strong amount of the variation in historical ride cost, making it suitable as the base prediction model for the RL pricing simulation.

metric		value
0	MAE	55.578
1	RMSE	74.427
2	R2	0.848

10 rows × 3 cols										Static Output		
	actual_historical_cost	predicted_historical_cost	absolute_error									
0	470.27	385.61	84.66									
1	286.41	309.67	23.26									
2	552.27	649.52	97.26									
3	267.74	338.66	70.92									
4	111.11	121.29	10.18									
5	359.13	398.67	39.54									
6	173.89	178.36	4.47									
7	196.32	193.20	3.12									
8	555.40	398.01	157.39									
9	163.21	134.52	28.70									

Figure 4: Random Forest evaluation metrics and prediction sample

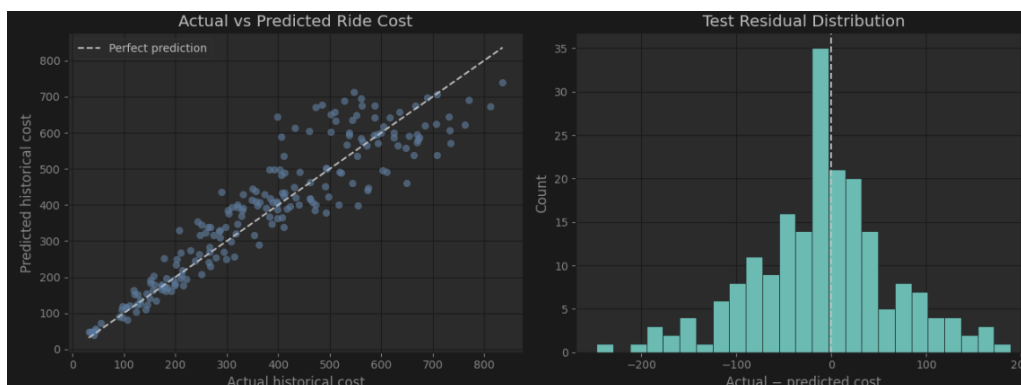


Figure 5: Random Forest actual vs predicted ride cost and residual distribution

The fixed-pricing baseline is simple and stable because it always keeps the multiplier at 1.0. Since it's fixed, it does not adapt to changing market conditions. This means it may miss opportunities to increase revenue when demand is high or reduce prices when demand is low.

The rule-based pricing method is more flexible than fixed pricing because it changes the multiplier based on the demand-supply ratio. This allows it to respond to busy or quiet market conditions. But, its still limited because the pricing rules are manually defined and do not improve through learning.

The RL pricing method uses the trained Q-learning policy to select a multiplier based on the current market state. This allows the agent to make more adaptive pricing decisions than the fixed baseline. The Q-learning agent was trained for 180 episodes. The final Q-table contained 730 states, and the final epsilon value was 0.050, showing that exploration reduced over time and the agent mainly used its learned policy by the end of training. On the test set, only 35 out of 200 states were already known from training, so the model used the fallback multiplier of 1.1 for unseen states.



Figure 6: Q-learning training rewards over episodes

Three pricing strategies were then compared: fixed pricing, rule-based pricing and RL pricing. Fixed pricing produced total expected revenue of 77,757.11 with an average reward of 388.79. Rule-based pricing produced the highest total expected revenue of 78,037.71, but its average reward was lower at 364.23. RL pricing produced total expected revenue of 77,451.24 with an average reward of 384.44.

	method	total_revenue	average_reward	average_multiplier	average_retained_demand
0	Fixed pricing	77,760.41	388.80	1.000	1.000
1	Rule-based pricing	78,041.03	364.24	1.183	0.849
2	RL pricing	77,728.37	386.36	1.084	0.925

Figure 7: Pricing policy comparison table

The results show that rule-based pricing generated the most revenue, but fixed pricing achieved the highest average penalised reward. This happened because the reward function includes a penalty for excessive pricing. The rule-based method used a higher average multiplier of 1.183, which increased revenue but also increased the penalty. Fixed pricing avoided this penalty by always using a multiplier of 1.0.

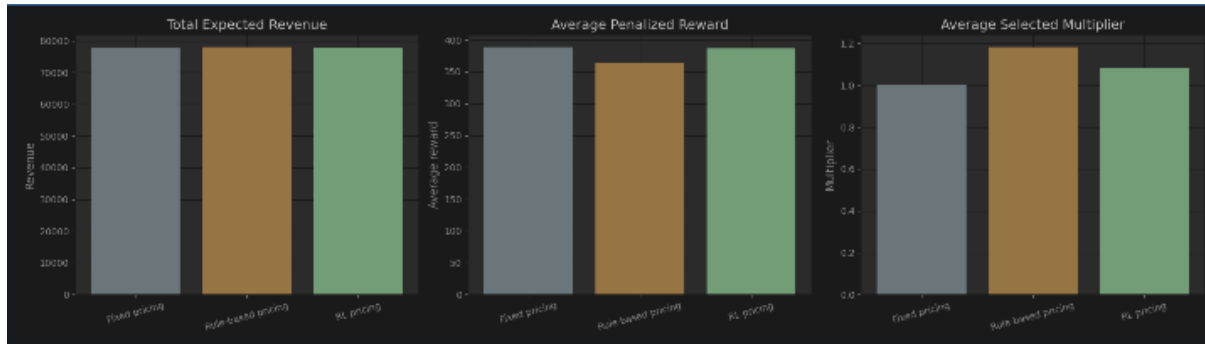


Figure 8: Pricing policy visual comparison

The RL pricing method used an average multiplier of 1.080, meaning it learned a slightly higher pricing strategy than fixed pricing but was less aggressive than rule-based pricing. RL did not outperform the baselines in this run, it still demonstrated how reinforcement learning can be applied to dynamic pricing decisions. The result suggests that the RL model may need further tuning, better state grouping, more training episodes or a more realistic demand-response model before it can outperform simpler pricing strategies.

Limitations and Conclusion

This project has some limitations. The main limitation is that the pricing system is based on a simulation rather than a live ridesharing platform. The dataset shows historical ride information, but it does not show how real customers would react if the price was changed. Because of this, customer demand response had to be estimated using the reward function and retained-demand assumptions.

Another limitation is that the Random Forest model predicts expected base ride cost, not real future demand. This means the environment is useful for testing pricing decisions, but it does not fully capture real market behaviour. The Q-learning model also used discretised state bins, which simplified the market state but may have caused some detail to be lost. This can explain why many test states were unseen and required the fallback multiplier.

The results showed that rule-based pricing achieved the highest total expected revenue, while fixed pricing achieved the highest average penalised reward. RL pricing did not outperform both baselines in this version of the experiment, but it still showed how Reinforcement Learning can be used to build an adaptive pricing strategy. The RL model selected a moderate average multiplier, meaning it was less aggressive than rule-based pricing but more flexible than fixed pricing.

Future improvements could include using real booking acceptance data, stronger demand prediction, more training episodes, better state grouping, price caps, fairness constraints and customer satisfaction measures. The model could also be tested across other datasets, such as hotel booking or retail pricing data, to check whether the same RL approach works in different business contexts.

This project demonstrates that dynamic pricing can be modelled as a Reinforcement Learning decision-making problem. The current RL model needs further tuning, the approach provides a useful foundation for revenue optimisation because it allows pricing decisions to adapt to changing demand and supply conditions.